



دانشگاه صنعتی امیرکبیر
(پلی تکنیک تهران)
دانشکده ریاضی و علوم کامپیوتر

پروژه اول هوش مصنوعی
رشته علوم کامپیوتر

Othello

نگارش

محمدحسین رستم‌دار

استاد درس

جناب آقای دکتر مهدی قطعی

استاد کارگاه

جناب آقای بهنام یوسفی مهر

تاریخ

آبان ۱۴۰۳

چکیده

این پروژه برای پیاده سازی بازی Othello می باشد که روی یک تخته 8×8 بین دو بازیکن انجام می شود. هر بازیکن باید مهره های حریف را بین مهره های خود محاصره کرده و رنگ آن ها را تغییر دهد. در پایان بازی، بازیکنی که بیشترین مهره را در تخته داشته باشد، برنده است.

که در آن از الگوریتم **Minimax** استفاده شده است که با بررسی تمام حالات ممکن، بهترین حرکت را برای بازیکن انتخاب می کند و برای بهینه سازی الگوریتم **Minimax** و افزایش سرعت جستجو، از هرس آلفا-بتا استفاده می شود که شاخه های غیرضروری درخت را حذف می کند. همچنین از الگوریتم **Expectimax** بهره گرفته شده است که نسخه ای پیشرفته از **Minimax** است که عناصر تصادفی را در نظر می گیرد و برای شرایطی مانند حرکات تصادفی بازیکن ها به کار می رود و از تابع ارزیابی برای تعیین سودمندی یا ارزشیابی وضعیت های مختلف در حین بازی و انتخاب بهترین حرکت استفاده می شود و شامل توضیحاتی از فایل ها و توابع اصلی مانند فایل های `board.py` برای مدیریت تخته، `minimax.py` برای پیاده سازی الگوریتم مینیمکس، و `game.py` برای مدیریت نوبت ها و اجرای بازی است.

واژه های کلیدی:

الگوریتم های **Expectimax** , **Minimax** و هرس آلفا-بتا , تابع ارزیابی

صفحه

فهرست مطالب

چکیده.....	۴
فصل اول مقدمه.....	3
فصل دوم توضیح کلی از بازی و نحوه ای که بازی پیاده سازی شده.....	5
۱-۲- توضیح کلی بازی.....	6
۲-۲- توضیح الگوریتم های پیاده سازی شده.....	۷
۳-۲- تحلیل عمق درخت.....	۱۰
فصل سوم تعریف تابع ارزیابی و علت تعریف به این صورت.....	۱۴
فصل چهارم نتیجه گیری و جمع بندی.....	۲۶
منابع و مراجع.....	۲۸

فصل اول

مقدمه

مقدمه:

در این مقاله، تلاش شده است تا با استفاده از الگوریتم‌های هوش مصنوعی و تکنیک‌های جستجوی بهینه، راهکاری برای بازی تخته‌ای "اتلو" ارائه شود. اتلو یک بازی استراتژیک دو نفره است که هدف آن، محاصره و تبدیل مهره‌های حریف به رنگ خودی است. بازیکنی که در پایان بازی بیشترین مهره را در اختیار داشته باشد، برنده می‌شود. این ویژگی‌ها، اتلو را به بستری مناسب برای ارزیابی عملکرد الگوریتم‌های هوش مصنوعی در تصمیم‌گیری‌های پیچیده تبدیل کرده است.

برای شبیه‌سازی حرکات بازیکنان و انتخاب بهینه‌ترین حرکت، از الگوریتم‌های Minimax و Expectimax استفاده شده است. Minimax با بررسی تمام حالات ممکن، حرکتی را انتخاب می‌کند که بدترین نتایج را برای حریف به حداقل برساند، در حالی که Expectimax رویکردی تصادفی‌تر را در مواجهه با حرکات غیرقابل پیش‌بینی در نظر می‌گیرد. همچنین، برای افزایش کارایی و سرعت الگوریتم‌ها، از هرس آلفا-بتا به منظور حذف شاخه‌های غیرضروری استفاده شده است.

این پروژه با بهره‌گیری از یک تابع ارزیابی جامع، وضعیت‌های مختلف بازی را از نظر مهره‌های ثابت، کنترل خانه‌های استراتژیک و قابلیت حرکت بازیکنان تحلیل می‌کند. این تابع به الگوریتم‌ها کمک می‌کند تا تصمیمات هوشمندانه‌تری بگیرند و عملکرد بهتری در بازی اتلو ارائه دهند.

فصل دوم

توضیح کلی از بازی و نحوه ای که بازی پیاده سازی شده

2-1- توضیح کلی بازی:

بازی اتلو (Othello)، که یک بازی تخته‌ای است که توسط دو بازیکن روی یک تخته 8×8 انجام می‌شود. هدف بازیکنان این است که در پایان بازی بیشترین تعداد مهره‌ها را به رنگ خود درآورند. بازیکنان با قرار دادن مهره‌های جدید و محاصره کردن مهره‌های حریف، مهره‌های بین مهره‌های خود را به رنگ خود برمی‌گردانند.

پیاده‌سازی این پروژه شامل فایل‌های متعددی است که هر کدام مسئولیت خاصی در بازی دارند. در ادامه به بررسی هر فایل می‌پردازیم:

- **board.py** : این فایل مسئول تعریف و مدیریت تخته بازی است. توابع مربوط به چیدمان مهره‌ها، بررسی وضعیت تخته، و مکان‌های ممکن برای حرکت بازیکنان در اینجا قرار دارد.

متدها:

`reset`: صفحه‌ی بازی را با چیدمان اولیه اتلو مقداردهی می‌کند.

`place_piece`: یک مهره روی صفحه قرار می‌دهد و مهره‌های حریف را بر اساس قوانین بازی برمی‌گرداند.

`is_valid_move`: بررسی می‌کند که آیا حرکت برای یک بازیکن معتبر است یا خیر.

`get_valid_moves`: لیستی از حرکات معتبر برای یک بازیکن را برمی‌گرداند.

`score`: امتیاز کنونی هر بازیکن را محاسبه و باز می‌گرداند.

- **game.py**: این فایل منطق اصلی بازی اتلو را شامل می‌شود. وضعیت‌های مختلف بازی، قوانین حرکت، و پایان بازی در این فایل پیاده‌سازی شده‌اند.

متدها:

start: یک بازی جدید را شروع کرده و صفحه را تنظیم می کند.

make_move: حرکتی را برای بازیکن کنونی انجام داده و نوبت را عوض می کند.

is_game_over: بررسی می کند که آیا بازی تمام شده است (هیچ حرکت معتبری برای هیچ یک از بازیکنان وجود ندارد).

get_winner: بر اساس امتیازها، برنده را تعیین می کند.

- **player.py:** این فایل بازیکنان را مدیریت می کند و بسته به نوع بازیکن، می تواند بازیکنان انسانی، بازیکنان با الگوریتم های هوش مصنوعی مانند Minimax و Expectimax، و یا بازیکنان تصادفی را پیاده سازی کند.

متدها:

get_move: حرکت بازیکن را دریافت می کند (به صورت دستی یا توسط هوش مصنوعی).

is_ai: بررسی می کند که آیا بازیکن هوش مصنوعی است یا انسان.

- **main.py:** نقطه ورود بازی است و به احتمال زیاد شامل تنظیمات اولیه و آغاز اجرای بازی می باشد.

- **expectimax.py و minimax.py:** این دو فایل الگوریتم های هوش مصنوعی را پیاده سازی می کنند. الگوریتم Minimax با توجه به حرکات بازیکن و حریف بهینه ترین حرکت را انتخاب می کند Expectimax. نیز به منظور ایجاد حرکات احتمالی و تصادفی برای افزایش چالش و هوشمندی بازی استفاده می شود.

- **utils.py:** فایل حاوی توابع کمکی و ابزارهای مورد نیاز برای انجام عملیات های عمومی در بازی است.

هر کلاس و متد آن ها یک تنظیم ماژولار را تشکیل می دهند که در آن مدیریت بازی، صفحه و تصمیم گیری هوش مصنوعی به طور مستقل عمل می کنند و این موضوع باعث می شود آزمایش و گسترش آسان تر شود.

۲-۲- توضیح الگوریتم های پیاده سازی شده:

الگوریتم Minimax

1 تابع minimax

این تابع، وظیفه اجرای الگوریتم Minimax را برای یک سطح (lvl) از بازی بر عهده دارد. ورودی‌های این تابع عبارتند از:

- state: وضعیت فعلی تخته بازی.
- Lvl: عمق جستجوی Minimax چند حرکت جلوتر را می‌بیند.
- is_maximizing: تعیین می‌کند که آیا بازیکن در حال حداکثر کردن امتیاز خود است یا در حال کمینه‌سازی امتیاز حریف.
- Color: رنگ بازیکن فعلی (سیاه یا سفید).
- pruning_enabled: فعال‌سازی یا غیرفعال‌سازی هرس آلفا-بتا
- alpha_val و beta_val: مقادیر اولیه برای الگوریتم هرس آلفا-بتا

گام‌های اجرای تابع minimax

۱. شرط پایان جستجو:

- اگر lvl برابر ۰ باشد یا وضعیت فعلی تخته (بر اساس تابع is_terminal) به نقطه‌ی پایان بازی برسد، جستجو متوقف شده و امتیاز تخته با تابع evaluate_board برگردانده می‌شود.

۲. گرفتن حرکات معتبر:

- اگر lvl بیشتر از صفر باشد، حرکات‌های ممکن بازیکن با `get_valid_moves(color)` گرفته می‌شود. اگر حرکتی وجود نداشته باشد، این یعنی هیچ تغییری نمی‌تواند اعمال شود و تخته ارزیابی می‌شود.

۳. Maximizing Player:

- اگر نوبت بازیکن در حالت `is_maximizing` باشد (در حال حداکثرسازی)، بهترین امتیاز (`best_score`) برابر منفی بی نهایت قرار می گیرد و در هر حرکت معتبر، مراحل زیر اجرا می شود:

- **شبیه سازی حرکت:** یک نسخه ی کپی از وضعیت فعلی تخته (`simulated_board`) ایجاد و حرکت جاری روی آن اعمال می شود.
- **فراخوانی Minimax برای سطح بعدی:** با اعمال حرکت جدید، تابع `minimax` برای سطح `lvl - 1` و با تنظیم `is_maximizing=False` صدا زده می شود.
- **بروزرسانی بهترین امتیاز و حرکت بهینه:** اگر امتیاز برگردانده شده بهتر از `best_score` باشد، آن را به روز می کند و حرکت جاری را به عنوان بهترین حرکت (`optimal_move`) ذخیره می کند.
- **اعمال Alpha-Beta Pruning:** مقدار `alpha_val` با مقدار بالاترین امتیاز دیده شده بروز می شود. اگر `beta_val` کمتر یا مساوی `alpha_val` شود، جستجوی بقیه ی حرکات متوقف می شود (قطع شاخه ای که نیازی به بررسی بیشتر ندارد).

۴. Minimizing Player:

- اگر نوبت حریف باشد (`is_maximizing=False`)، امتیاز `worst_score` برابر بی نهایت قرار می گیرد. حریف فرضی رنگ متضاد `opponent` خواهد بود. در هر حرکت معتبر:
- **شبیه سازی حرکت:** حرکت فعلی در نسخه ای از تخته کپی (`simulated_board`) اعمال می شود.
- **فراخوانی Minimax برای سطح بعدی:** تابع `minimax` برای سطح `lvl - 1` و با تنظیم `is_maximizing=True` فراخوانی می شود.
- **بروزرسانی بدترین امتیاز و حرکت بهینه:** اگر امتیاز کوچکتر از `worst_score` باشد، آن را به روز می کند.
- **اعمال Alpha-Beta Pruning:** مقدار `beta_val` با مقدار پایین ترین امتیاز بروز می شود و اگر `beta_val` کمتر یا مساوی `alpha_val` شود، بقیه حرکات بررسی نمی شوند.

۵. خروجی تابع:

○ تابع `minimax` در نهایت بهترین حرکت (`optimal_move`) و امتیاز مربوط به آن را برمی گرداند.

تابع `evaluate_board`

این تابع امتیاز تخته را برای رنگ بازیکن جاری محاسبه می کند و چندین عامل کلیدی بازی را در نظر می گیرد:

- امتیاز خام بازیکن و حریف: اختلاف مهره های بازیکن با حریف را به دست می آورد.
- کنترل خانه های گوشه ای: (**Corner Advantage**) خانه های گوشه ها از اهمیت زیادی برخوردارند و به همین دلیل هر کدام از آن ها ۲۵ امتیاز به امتیاز اضافه می کند.
- کنترل خانه های لبه ای: (**Edge Control**) خانه های لبه ای (غیر از گوشه ها) در تخته را بررسی و به ازای هر مهره ۵ امتیاز اضافه می کند.
- قابلیت حرکت: (**Mobility**) تفاوت تعداد حرکات ممکن بازیکن و حریف را با ضریب ۱۰ در امتیاز دخیل می کند.
- مهره های ثابت: (**Stable Discs**) مهره های ثابت (غیرقابل برگشت) برای بازیکن و حریف را با ضریب ۱۵ در امتیاز محاسبه می کند.

توابع `is_stable` و `count_stable_discs`

این توابع برای شناسایی و شمارش مهره های ثابت بازیکن استفاده می شوند:

- تابع `count_stable_discs` تمامی خانه های تخته را برای بازیکن جاری مرور کرده و تعداد خانه های ثابت را شمارش می کند.
- تابع `is_stable` تعیین می کند که آیا مهره ای ثابت است یا نه. برای اینکار در تمام جهت های افقی، عمودی و مورب حرکت می کند. اگر خانه ای خالی در هر جهت پیدا شود، مهره پایدار نیست و تابع `False` برمی گرداند؛ در غیر این صورت، `True` برگردانده می شود.

این پیاده سازی به بازیکن کمک می کند تا با بررسی حرکات های مختلف، بهترین تصمیم ممکن را برای هر مرحله از بازی بگیرد و با استفاده از `Alpha-Beta Pruning` فرآیند جستجو را سریع تر و کاراتر کند.

الگوریتم Expectimax

این کد الگوریتم **Expectimax** را برای بازی اتلو پیاده سازی می کند. Expectimax. مشابه Minimax است، با این تفاوت که به جای کمینه سازی امتیاز حریف در هر سطح، از میانگین امتیازها برای حرکات حریف استفاده می کند. این رویکرد در مواقعی مفید است که رفتار حریف ممکن است تصادفی یا غیرقابل پیش بینی باشد.

در ادامه مراحل و جزئیات پیاده سازی این کد را توضیح می دهیم:

تابع expectimax

این تابع، الگوریتم Expectimax را برای یک سطح مشخص (depth) و برای بازیکن فعلی اجرا می کند. ورودی های تابع عبارتند از:

- Board: وضعیت فعلی تخته بازی.
- Depth: عمق جستجوی Expectimax (چند حرکت جلوتر را در نظر می گیرد).
- maximizing_player: مشخص می کند که آیا بازیکن در حال حداکثر کردن امتیاز است یا خیر.
- player_color: رنگ بازیکن جاری (سیاه یا سفید).

گام های اصلی تابع expectimax

۱. شرط توقف:

- اگر depth به صفر برسد یا هیچ حرکت ممکن برای بازیکن سیاه ('B') و سفید ('W') نباشد، جستجو متوقف شده و وضعیت فعلی تخته با استفاده از تابع evaluate_board ارزیابی می شود.

۲. حرکات معتبر:

- `get_valid_moves` برای گرفتن لیست حرکات ممکن بازیکن در حالت حداکثرسازی (Maximizing) یا حریف در حالت میانگین گیری (Expected) استفاده می شود. اگر حرکتی وجود نداشته باشد، وضعیت تخته ارزیابی می شود.

۳. حالت: Maximizing Player

- اگر نوبت بازیکن اصلی باشد (در حال حداکثرسازی)، بهترین امتیاز (`max_eval`) برابر منفی بی نهایت قرار می گیرد و بهترین حرکت (`best_move`) به `None` مقداردهی می شود.
- برای هر حرکت ممکن:
 - یک نسخه از تخته (`new_board`) با تابع `copy_board` ایجاد و حرکت جاری روی آن اعمال می شود.
 - تابع `expectimax` برای تخته جدید با یک سطح کمتر `depth-1` و با تغییر نوبت به حریف (`maximizing_player=False`) فراخوانی می شود.
 - اگر امتیاز به دست آمده از حرکت جاری بیشتر از `max_eval` باشد، مقدار آن به روز می شود و حرکت فعلی به عنوان بهترین حرکت ذخیره می شود.
- در نهایت، `best_move` و `max_eval` به عنوان خروجی این بخش برگردانده می شوند.

۴. حالت میانگین گیری: (Expected Value)

- اگر نوبت حریف باشد (در حالت میانگین گیری)، یک متغیر `expected_value` با مقدار اولیه صفر تعریف می شود.
- برای هر حرکت ممکن:
 - یک نسخه از تخته ساخته شده و حرکت حریف روی آن اعمال می شود.
 - تابع `expectimax` برای سطح `depth-1` و با تنظیم `maximizing_player=True` صدا زده می شود.
 - امتیاز به دست آمده از حرکت جاری به `expected_value` اضافه می شود.
- در نهایت `expected_value` تقسیم بر تعداد حرکات ممکن می شود تا مقدار متوسط (Expected Value) برگردانده شود.

تابع `evaluate_board`

این تابع برای ارزیابی وضعیت فعلی تخته به کار می رود. به طور کلی معیارهای زیر را در نظر می گیرد:

- امتیاز خام بازیکن و حریف : اختلاف تعداد مهره های بازیکن از حریف.
- کنترل خانه های گوشه ای : خانه های گوشه ای که از اهمیت بالایی برخوردارند، اگر متعلق به بازیکن باشند، امتیاز بیشتری خواهند داشت (هر کدام ۲۵ امتیاز).
- کنترل خانه های لبه ای : خانه های لبه ای به ازای هر مهره ۵ امتیاز به امتیاز نهایی اضافه می کند.
- قابلیت حرکت (Mobility) : اختلاف تعداد حرکات ممکن بازیکن و حریف را با ضریب ۱۰ در امتیاز نهایی دخیل می کند.
- مهره های ثابت (Stable Discs) : تعداد مهره های غیرقابل برگشت بازیکن و حریف با ضریب ۱۵ محاسبه می شود.

توابع `is_stable` و `count_stable_discs`

این توابع برای محاسبه تعداد مهره های ثابت بازیکن استفاده می شوند:

- `count_stable_discs` : با استفاده از تابع `is_stable`، تمامی خانه های تخته را برای بازیکن جاری بررسی کرده و تعداد مهره های ثابت را شمارش می کند.
- `is_stable` : این تابع بررسی می کند که آیا یک مهره ثابت است یا نه. مهره ثابت، مهره ای است که توسط مهره های دیگر در همه جهتها (عمودی، افقی و مورب) محصور شده باشد و دیگر نمی تواند توسط حریف تغییر کند.

تابع `copy_board`

این تابع یک کپی عمیق از تخته ایجاد می کند تا وضعیت جدید تخته برای هر حرکت شبیه سازی شود بدون اینکه روی تخته اصلی تغییری اعمال گردد.

در این پیاده سازی از **Expectimax**، بازیکن سعی می کند بهترین حرکت ممکن را انتخاب کند، در حالی که رفتار حریف به صورت تصادفی در نظر گرفته می شود و میانگین امتیاز حرکتهای ممکن او محاسبه می گردد.

۲-۳- تحلیل عمق درخت:

۱. تأثیر عمق جستجو (depth) بر کیفیت تصمیم‌گیری

عمق جستجو نشان می‌دهد که الگوریتم تا چند مرحله به جلو حرکت‌ها را پیش‌بینی می‌کند. هرچه عمق بیشتر باشد، تعداد حرکات بیشتری بررسی می‌شود و الگوریتم قادر به پیش‌بینی دقیق‌تری از پیامدهای هر حرکت خواهد بود. با این حال، افزایش عمق جستجو، باعث افزایش پیچیدگی محاسباتی و زمان اجرای الگوریتم می‌شود، زیرا تعداد حالات ممکن به صورت نمایی افزایش می‌یابد.

- **عمق ۲:** در این سطح، الگوریتم به دنبال بهترین حرکت برای بازیکن خودی و پاسخ فوری بازیکن حریف است. این جستجو به الگوریتم اجازه می‌دهد که حرکات نزدیک آینده را بسنجد، اما عمق کمی دارد و ممکن است به تصمیمات بهینه‌تری منجر نشود، چرا که از پیامدهای بلندمدت غافل می‌ماند.
- **عمق ۳:** در این عمق، الگوریتم می‌تواند حرکات حریف و پاسخ‌های احتمالی خودی را بررسی کند. این باعث می‌شود که الگوریتم، دید وسیع‌تری از بازی داشته باشد و تصمیمات هوشمندانه‌تری بگیرد. اما، این افزایش عمق می‌تواند زمان محاسبات را به‌ویژه در صورت نبود هرس آلفا-بتا (alpha-beta pruning) به‌طور قابل توجهی بیشتر کند.

۲. مقایسه عملکرد در عمق‌های مختلف

- **عمق ۲ (کمتر عمیق):**
 - مزایا: سریع‌تر اجرا می‌شود و می‌تواند در زمان واقعی پاسخ دهد.

- معایب: چون پیامدهای بلندمدت را نمی بیند، ممکن است حرکتهایی انتخاب شود که در ادامه به ضرر بازیکن باشد.
- **عمق ۳ (بیشتر عمیق):**
 - مزایا: الگوریتم حرکات بیشتری را بررسی می کند و می تواند استراتژی هایی با پایداری بیشتر انتخاب کند، که معمولاً به نتایج بهتری منجر می شود.
 - معایب: زمان اجرای بیشتری دارد، به ویژه در تخته هایی که حرکات متعددی در هر مرحله امکان پذیر است.

تحلیل عملکرد دو الگوریتم Minimax و Expectimax در عمق ۲ و ۳

الگوریتم Minimax با Alpha-Beta Pruning

Minimax با فرض اینکه حریف منطقی ترین حرکت ممکن را انجام خواهد داد، کار می کند. در سطح های `is_maximizing=True` به حداکثرسازی امتیاز بازیکن و در سطح های `is_maximizing=False` به حداقل سازی امتیاز حریف می پردازد.

• عمق ۲:

- در این عمق، Minimax حرکات ممکن بازیکن را در سطح اول و حرکات ممکن حریف را در سطح دوم بررسی می کند.
- در این سطح، الگوریتم به دنبال حرکات بازیکن می رود که بدترین حالت ممکن برای او را بهتر کنند. به این ترتیب، امکان محاسبه دقیق ترین حرکت برای بازیکن فراهم است، اما این تحلیل تنها یک حرکت حریف را در نظر می گیرد که ممکن است به درستی تمامی حرکات احتمالی او را ارزیابی نکند.
- Alpha-Beta Pruning کمک می کند که بعضی شاخه های بررسی حذف شده و زمان محاسبات کاهش یابد.

• عمق ۳:

- در این عمق، Minimax در اولین سطح، حرکات بازیکن، در دومین سطح، حرکات حریف، و در سومین سطح دوباره حرکات بازیکن را ارزیابی می کند.
- این سطح از عمق، دقت تحلیل را افزایش می دهد زیرا حالا می توان یک دور حرکت حریف و یک دور بازگشت بازیکن به بازی را شبیه سازی کرد.

- با توجه به پیچیدگی زمانی $O(b^d)$ که در آن b تعداد حرکات ممکن و d عمق است، اجرای الگوریتم با عمق ۳ نسبت به عمق ۲ زمان بیشتری نیاز دارد، اما Alpha-Beta Pruning کمک می کند تا برخی شاخه ها زودتر حذف شوند و محاسبات بهینه تر شوند

الگوریتم Expectimax

Expectimax بر اساس فرض تصادفی بودن رفتار حریف عمل می کند و به جای حداقل سازی امتیاز حریف، از میانگین گیری امتیاز حرکات ممکن حریف استفاده می کند. در اینجا حریف همیشه بهترین حرکت ممکن را انتخاب نمی کند و احتمال تصمیم گیری تصادفی او در نظر گرفته می شود.

• عمق ۲:

- در این عمق، Expectimax مانند Minimax ابتدا حرکات بازیکن و سپس حرکات حریف را بررسی می کند. با این تفاوت که به جای انتخاب کمترین امتیاز، امتیازها را میانگین می گیرد.
- این سطح از عمق برای بازی هایی با حریف تصادفی تر می تواند مفید باشد، زیرا انتظار متوسط نتیجه حرکات حریف در نظر گرفته می شود و الگوریتم نمی تواند در برابر یک حریف بسیار منطقی و باتجربه به اندازه Minimax دقیق عمل کند.
- پیچیدگی محاسباتی این سطح مشابه Minimax است، اما به دلیل محاسبه میانگین به جای کمینه، الگوریتم ممکن است مقداری زمان بیشتری نسبت به Minimax برای ارزیابی نیاز داشته باشد.

• عمق ۳:

- در این عمق، Expectimax مانند Minimax، حرکات بازیکن و حریف را در سه سطح بررسی می کند. اما در سطح حریف، امتیازات میانگین گرفته می شوند.
- این عمق برای حالتی که رفتار حریف غیرقابل پیش بینی باشد مفید است. چرا که الگوریتم Expectimax به جای ارزیابی همه حرکات ممکن، تنها میانگین آن ها را در نظر می گیرد. بنابراین، به ویژه در بازی هایی که حریف دارای الگوی حرکتی مشخص نیست، کارآمدتر از Minimax خواهد بود.
- از لحاظ محاسباتی، این الگوریتم نیاز به بررسی تعداد بیشتری حرکت در هر سطح دارد و به علت محاسبه میانگین در سطح های عمق بالا، زمان بیشتری نسبت به Minimax مصرف می کند

این تحلیل نشان می دهد که اگر رفتار حریف منطقی باشد، Minimax با عمق ۳ انتخاب بهتری است، اما اگر حریف رفتار تصادفی داشته باشد، Expectimax می تواند به دلیل میانگین گیری در ارزیابی حرکات، نتایج بهتری بدهد.

در نتیجه، در عمق ۲ و ۳، الگوریتم Expectimax معمولاً کندتر از Minimax خواهد بود، چرا که شاخه های اضافی را هرس نمی کند.

فصل سوم

تعریف تابع ارزیابی و علت تعریف به این صورت

تابع **evaluate_board** برای ارزیابی وضعیت فعلی تخته بازی اتلو برای بازیکن مشخص (player_color) طراحی شده است. هدف این تابع محاسبه امتیاز تخته به صورتی است که بتواند میزان مطلوبیت وضعیت تخته را برای بازیکن مشخص شده بسنجد. این امتیاز در تصمیم‌گیری‌های استراتژیک و انتخاب بهترین حرکت توسط الگوریتم‌هایی مانند Minimax یا Expectimax استفاده می‌شود.

- محاسبه امتیاز خام بازیکن و حریف:

```
B_score, W_score = board.get_score()
opponent_color = 'w' if player_color == 'B' else 'B'
player_score = B_score if player_color == 'B' else W_score
opponent_score = W_score if player_color == 'B' else B_score

score = player_score - opponent_score
```

ابتدا امتیاز (تعداد مهره‌های) بازیکن و حریف محاسبه می‌شود. سپس اختلاف بین تعداد مهره‌های بازیکن و حریف به عنوان امتیاز پایه‌ی تخته تنظیم می‌شود.

- برتری خانه‌های گوشه‌ای (Corner Advantage) :

```
corners = [(0, 0), (0, board.size - 1), (board.size - 1, 0), (board.size - 1, board.size - 1)]
player_corners = sum(1 for x, y in corners if board.board[x][y] == player_color)
opponent_corners = sum(1 for x, y in corners if board.board[x][y] == opponent_color)
score += 25 * (player_corners - opponent_corners)
```

خانه‌های گوشه‌ای اهمیت زیادی دارند، زیرا وقتی مهره‌ای در آن‌ها قرار گیرد، حریف نمی‌تواند آن را برگرداند. به ازای هر خانه گوشه‌ای که بازیکن در اختیار دارد، ۲۵ امتیاز به امتیاز نهایی اضافه می‌شود و به ازای خانه‌های گوشه‌ای که حریف در اختیار دارد، از امتیاز بازیکن کم می‌شود.

• کنترل خانه‌های لبه‌ای (Edge Control) :

```
edges = [
    (0, i) for i in range(1, board.size - 1)
] + [(board.size - 1, i) for i in range(1, board.size - 1)] + [
    (i, 0) for i in range(1, board.size - 1)
] + [(i, board.size - 1) for i in range(1, board.size - 1)]
player_edges = sum(1 for x, y in edges if board.board[x][y] == player_color)
opponent_edges = sum(1 for x, y in edges if board.board[x][y] == opponent_color)
score += 5 * (player_edges - opponent_edges)
```

خانه‌های لبه‌ای نسبت به خانه‌های میانی تخته اهمیت بیشتری دارند اما به اندازه خانه‌های گوشه‌ای مهم نیستند. برای هر خانه لبه‌ای که بازیکن کنترل می‌کند، ۵ امتیاز به امتیاز نهایی افزوده می‌شود و برای هر خانه لبه‌ای تحت کنترل حریف ۵ امتیاز کم می‌شود.

• قابلیت حرکت (Mobility) :

```
player_moves = len(board.get_valid_moves(player_color))
opponent_moves = len(board.get_valid_moves(opponent_color))
if player_moves + opponent_moves != 0:
    score += 10 * (player_moves - opponent_moves)
```

Mobility به تعداد حرکات ممکن هر بازیکن اشاره دارد و نشان‌دهنده انعطاف‌پذیری بازیکن در حرکات‌های آینده است. هرچه بازیکن حرکات بیشتری داشته باشد، وضعیت بهتری دارد. اختلاف تعداد حرکات ممکن بازیکن و حریف با ضریب ۱۰ در امتیاز نهایی تأثیرگذار است.

• مهره‌های ثابت (Stable Discs) :

```
player_stable_discs = count_stable_discs(board, player_color)
opponent_stable_discs = count_stable_discs(board, opponent_color)
score += 15 * (player_stable_discs - opponent_stable_discs)
```

مهره‌های ثابت به مهره‌هایی اشاره دارد که دیگر توسط حریف قابل تغییر نیستند. این مهره‌ها، به‌ویژه در نزدیکی خانه‌های گوشه‌ای، امتیاز بالایی دارند زیرا تا پایان بازی پابرجا می‌مانند. اختلاف تعداد مهره‌های ثابت بازیکن و حریف با ضریب ۱۵ به امتیاز نهایی اضافه می‌شود.

علت تعریف تابع به این شکل

تابع `evaluate_board` به گونه‌ای طراحی شده که معیارهای اصلی و تاثیرگذار در بازی اتلو را در خود جای دهد و وضعیت تخته را از زوایای مختلفی بسنجد. به دلیل ماهیت چندوجهی بازی اتلو، تنها بررسی تعداد مهره‌ها نمی‌تواند معیار مناسبی برای ارزیابی باشد؛ برای همین از معیارهای دیگری نظیر:

- **کنترل خانه‌های گوشه‌ای و لبه‌ای**: که نشان‌دهنده پایداری و قدرت موقعیت بازیکن هستند.
- **قابلیت حرکت**: که نشان‌دهنده‌ی انعطاف‌پذیری و گزینه‌های حرکت است.
- **مهره‌های ثابت**: که اهمیت بسیار زیادی در تعیین نتیجه نهایی دارند.

استفاده شده است. این طراحی باعث می‌شود تابع بتواند ارزیابی دقیق‌تری از وضعیت تخته داشته باشد و در نتیجه، الگوریتم‌هایی مثل Minimax و Expectimax بتوانند تصمیمات بهتری بر اساس نتایج این تابع بگیرند.

فصل چهارم

نتیجه گیری و جمع بندی

نتیجه گیری و جمع بندی:

این مقاله به بررسی و پیاده سازی هوش مصنوعی برای بازی تخته ای اتلو پرداخته است. با استفاده از الگوریتم های Expectimax ، Minimax و تکنیک هرس آلفا-بتا، سعی شده تا بهترین حرکات در بازی برای بازیکنان شبیه سازی شود. الگوریتم Minimax با تمرکز بر بهینه سازی حرکت بازیکن و الگوریتم Expectimax برای حرکات تصادفی در بازی به کار رفته است. این الگوریتم ها با ارزیابی وضعیت های مختلف بازی و محاسبه سودمندی هر حرکت، به بهبود عملکرد و سرعت جستجوی هوش مصنوعی کمک می کنند.

در این پروژه، ترکیبی از معیارها مانند تعداد مهره های ثابت، کنترل خانه های گوشه ای و لبه ای، و قابلیت حرکت به عنوان ورودی های تابع ارزیابی در نظر گرفته شده اند. این معیارها، علاوه بر بهبود دقت ارزیابی وضعیت، به الگوریتم ها کمک می کنند تا تصمیمات هوشمندانه تری اتخاذ کنند. این پروژه در نهایت موفق شده است با استفاده از ساختارهای ماژولار و بهینه سازی ها، استراتژی مناسبی برای اجرای هوش مصنوعی در بازی اتلو فراهم کند.

منابع و مراجع

<https://en.wikipedia.org/wiki/Reversi>

<https://berkeleyai.github.io/cs188-website/assets/notes/n3.pdf>

https://courses.cs.washington.edu/courses/cse573/04au/Project/mini1/RUSSIA/miniproject1_vaishu_muthu/Paper/Final_Paper.pdf

همچنین ارجاع می‌دهم به علیرضا مختاری / ۴۰۱۱۳۰۲۵